CS5229 Advanced Computer Networks

Programming Assignment – Background Briefing

Semester 1, AY2026/27

1 Introduction

This background briefing introduces the technical context needed for the programming assignment. The goal is not to provide a comprehensive treatment of distributed machine learning or datacenter networking, but to establish the key concepts required to understand the workload and reason about its network behavior.

We will begin with a brief introduction to distributed machine learning and collective communication, with particular focus on Ring AllReduce. We will then describe how the example collective workload is executed in our simulation environment using ASTRA-Sim and NS-3, and introduce the network mechanisms that are most relevant to the project, including RDMA transport, congestion control, and load balancing.

Finally, we will bring these components together in the workload used in this assignment. We will examine how the logical communication pattern is mapped onto the physical datacenter network, derive an idealized estimate of its execution time, and compare this expectation with the behavior observed in simulation.

The purpose of this document is to provide the foundation for your own investigation. You are not expected to become an expert in every component described here, but you should understand the system well enough to observe its behavior, reason about possible performance bottlenecks, and develop well-motivated hypotheses and solutions.

2 Background Knowledge

Before examining the workload used in this assignment, we first introduce several background concepts in distributed machine learning and datacenter networking.

2.1 A Small Bite of Distributed Machine Learning

At a high level, training a deep neural network repeatedly performs two main computational phases: *forward propagation*, where input data passes through the model to produce predictions, and *backward propagation*, where gradients are computed and used to update the model parameters.

As machine learning workloads grow, training is often distributed across multiple GPUs or servers. This is necessary because a model can be too large for a single device, or simply because using multiple devices can significantly increase training speed. Once computation is distributed, however, the participating devices must also exchange data during training.

Different ways of partitioning the training workload give rise to different forms of parallelism:

- **Data parallelism**, where each worker holds a copy of the model but processes a different batch of training data. The workers periodically exchange and aggregate gradients to keep their model states consistent.
- **Model parallelism**, where different parts of a model are placed on different devices, requiring intermediate data to be exchanged during model execution (including both forward and backward propagation).

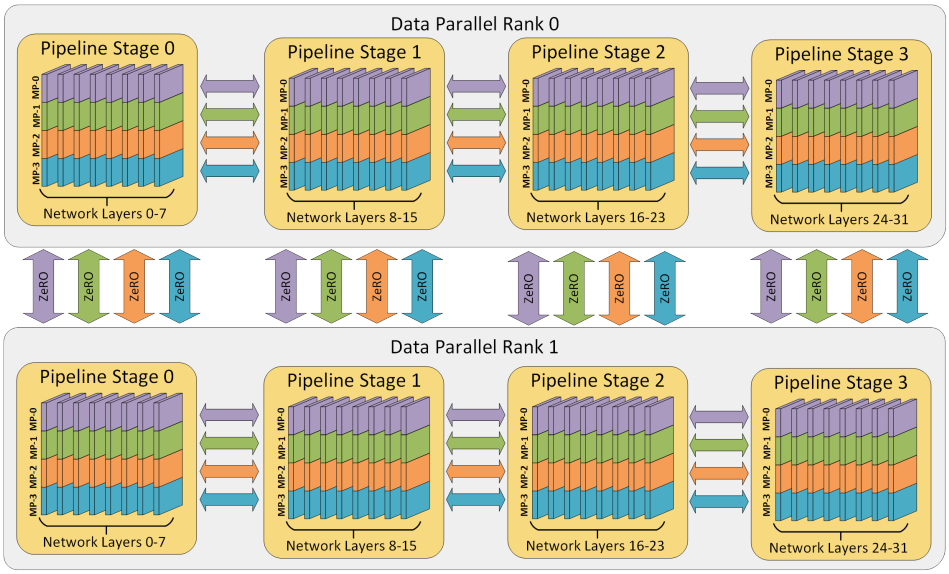


Figure 1: An example of combining pipeline parallelism and data parallelism in distributed training [4].

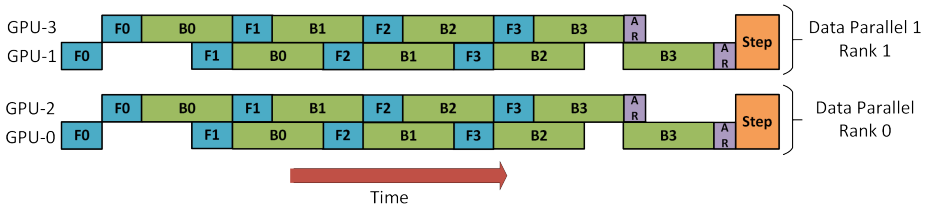


Figure 2: An example execution schedule for pipeline-parallel training with multiple data-parallel workers [4].

- **Pipeline parallelism**, where the model is divided into stages and different micro-batches progress through these stages in a pipelined manner.

Regardless of how training is parallelized, distributing computation introduces communication. Workers may need to aggregate gradients, exchange model states, or transfer intermediate activations and gradients between devices. As models and training clusters grow, these data transfers can become a significant part of the overall training time.

To perform such communication efficiently, distributed training systems make extensive use of *collective communication operations*, which coordinate structured data exchanges among a group of participating devices.

2.2 Collective Communication

In distributed machine learning, communication between participants often follows well-defined patterns known as *collective communication operations*. A collective operation coordinates data movement or computation across a group of participating processes. Common examples include *ReduceScatter*, *AllGather*, and *AllReduce*.

A collective operation describes *what* communication must take place, but not necessarily *how* it is carried out. The same collective can be implemented using different communication algorithms, such as trees, rings, or recursive-doubling schemes. These algorithms expose different trade-offs in latency, bandwidth utilization, and communication overhead.

Among these operations, AllReduce is particularly important for data-parallel training [9]. Each worker computes gradients from its local batch, and these gradients must be aggregated across workers

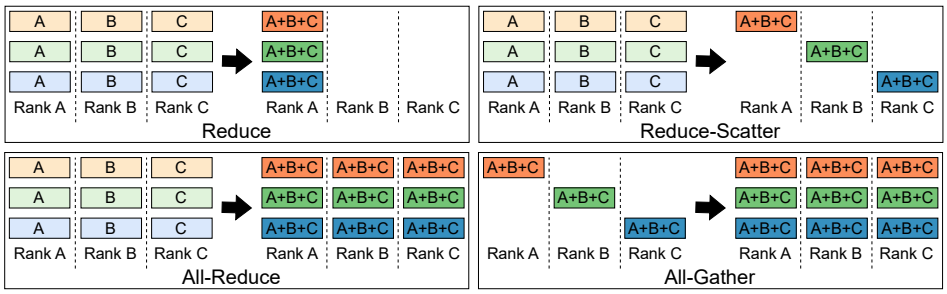


Figure 3: Examples of common collective communication operations.

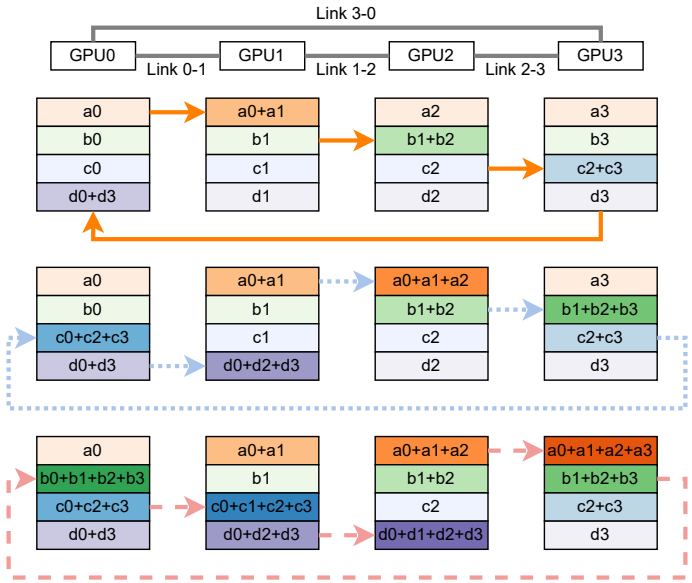


Figure 4: Example of a Ring-based Reduce-Scatter. An All-Reduce can be seen as Reduce-Scatter plus All-Gather.

so that they can update the model consistently. For large models, this communication can account for a significant fraction of the training time.

One widely used approach is *Ring AllReduce*, which organizes communication as a sequence of point-to-point transfers around a logical ring [7]. We therefore use Ring AllReduce as the main collective workload in this assignment.

2.3 Ring AllReduce

In Ring AllReduce, the participating nodes are **logically** arranged in a ring. Each node sends data to its next neighbor in the ring while receiving data from its previous neighbor. The word "logically" is important: two neighboring nodes in the ring do not necessarily correspond to neighboring machines in the physical network.

The input data is divided into N chunks for a ring of N participants. Ring AllReduce then proceeds in two phases. First, a *ReduceScatter* phase takes $N-1$ communication steps, during which the chunks are circulated and reduced so that each participant eventually holds one fully reduced chunk. This is followed by an *AllGather* phase of another $N-1$ steps, which circulates the reduced chunks until every participant has the complete result.

Therefore, a Ring AllReduce requires a total of $2(N-1)$ communication steps. During each step, all participants can send and receive concurrently, allowing the algorithm to make efficient use of available bandwidth without relying on a centralized coordinator.

The logical ring does not directly determine how packets travel through the physical network. In a

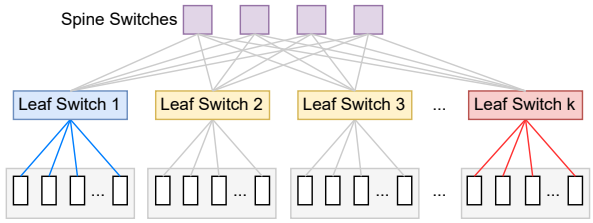


Figure 5: An example of a spine–leaf datacenter network topology.

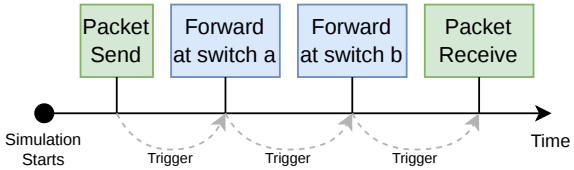


Figure 6: A simplified example of packet transmission represented as a sequence of discrete events in NS-3.

datacenter, hosts are connected through switches, and communication between two logical neighbors may traverse multiple network hops and share links with traffic from other pairs.

A common datacenter topology is the spine–leaf topology, illustrated in Fig. 5. Hosts are attached to leaf (or Top-of-Rack) switches, while multiple spine switches provide parallel paths between different groups of hosts.

Modern collective communication libraries can use multiple communication channels or rings concurrently to expose additional communication parallelism [7]. For this assignment, we use a simplified *multiple-1D-ring* workload inspired by this idea. Rather than modeling the full complexity of a production collective communication library, we construct several independent logical rings and execute them concurrently.

The exact placement of these rings onto the physical network will be introduced in Section 3. This mapping from *logical collective communication* to *physical network traffic* is central to the system behavior studied in this assignment.

2.4 Network Simulation

To study the network behavior of distributed training workloads, we use a simulation environment that combines **ASTRA-Sim** and **NS-3**. ASTRA-Sim generates the communication workload, while NS-3 simulates how the resulting traffic is transmitted through the network. This combination allows us to study packet-level effects such as congestion, queue buildup, packet loss, and path selection, while still preserving the communication dependencies of the distributed training workload.

2.4.1 Simulator Components

NS-3 NS-3 [3] is a discrete-event network simulator. Network activities such as packet transmission, forwarding, and reception are represented as events scheduled on a simulation timeline. For example, as illustrated in Fig. 6, a packet-send event at a host may trigger packet-forwarding events at intermediate switches, followed eventually by a packet-receive event at the destination. The simulator repeatedly processes the next scheduled event and may create additional events as the simulated network evolves.

Because packet transmission is simulated hop by hop, NS-3 can capture detailed interactions between traffic, queues, and network protocols. The simulation clock is independent of wall-clock time: a short interval of simulated network activity may require substantially more time to execute on the physical machine running the simulator. The NS-3 code base contains many components that are not directly relevant to this assignment. The repository README identifies the main files and functions that you are expected to inspect or modify.

ASTRA-Sim While NS-3 models the network itself, it does not determine what communication workload should be generated. ASTRA-Sim [1] provides this workload layer by modeling communication patterns from distributed machine learning.

In this assignment, we use ASTRA-Sim to generate the Ring AllReduce workload introduced earlier. ASTRA-Sim decomposes the collective operation into point-to-point message transfers and manages the dependencies between communication steps. These transfers are then passed to NS-3, which simulates their packet-level transmission through the physical network. This separation is useful to keep in mind: *ASTRA-Sim determines which participants communicate and when, while NS-3 determines how the resulting traffic behaves in the network.*

2.4.2 Transport

The point-to-point transfers generated by ASTRA-Sim are carried by an RDMA-based transport model in our simulated network, rather than TCP connections. RDMA (Remote Direct Memory Access) is commonly used in high-performance datacenter environments because it enables efficient data movement with low CPU involvement and low communication overhead. In our setup, the simulated transport follows a RoCEv2-style design, where RDMA traffic is carried over UDP/IP. Two packet-header fields are particularly useful when reading the simulator code:

- **UDP port number:** in our setup, the port number is used to distinguish different flows between a pair of hosts.
- **Sequence number:** packets within a flow carry sequence information that is used to track ordering and detect missing data.

Reliability and Packet Ordering The transport model expects packets within a flow to be delivered in order. If an out-of-order packet is observed, the simulated reliability mechanism uses Go-Back-N recovery: packets following the missing sequence are discarded, and the sender retransmits starting from the missing packet. As a result, packet reordering can introduce substantial retransmission overhead.

Packet loss can have a similar effect, since missing packets must also be retransmitted before the flow can make progress. These properties make transport behavior closely coupled with other parts of the network. For example, congestion, queue buildup, and path selection can affect packet delay, loss, and ordering, which in turn can affect the completion time of the collective communication workload.

2.4.3 Congestion Control

As discussed earlier, RDMA transport can be sensitive to packet loss and excessive queueing delay. Congestion control is therefore important for maintaining both high throughput and reliable communication. In our simulated network, congestion control is handled by DCQCN (Data Center Quantized Congestion Notification) [10].

DCQCN uses ECN (Explicit Congestion Notification) [6] to obtain congestion feedback from the network. Rather than waiting for a switch buffer to overflow and drop packets, a congested switch can mark packets with ECN. This congestion signal is eventually returned to the sender, allowing the sending RDMA NIC to reduce its transmission rate.

In our setup, ECN marking is determined by the queue length at a switch using a RED-style marking policy [8]. As illustrated in Fig. 7, packets are not marked while the queue length is below $K_{\min}$. Between $K_{\min}$ and $K_{\max}$, the marking probability increases with the queue length. Once the queue exceeds $K_{\max}$, packets are marked with probability 1.

This creates a feedback loop between the switches and the senders: growing queues lead to more congestion signals, and the senders respond by reducing their sending rates. Ideally, congestion should be detected early enough to prevent excessive queue buildup and packet loss, while avoiding unnecessary reductions in network utilization.

The behavior of this feedback loop depends on both the congestion-control algorithm and its configuration. For example, ECN thresholds that work well for one traffic pattern or network configuration may not necessarily behave the same way under another. When investigating the system, it may therefore be useful to examine how queue buildup, ECN marking, and sender rates evolve together over time.

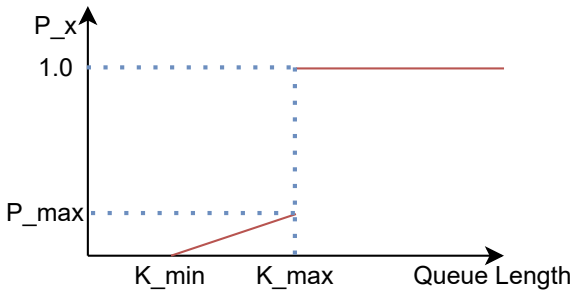


Figure 7: An example of a RED-style mapping from switch queue length to ECN marking probability.

2.4.4 Load Balancing

The spine–leaf topology introduced earlier provides multiple equal-cost paths between hosts in different pods. These parallel paths offer additional network capacity, but only if traffic can be distributed effectively across them. A load-balancing mechanism is therefore needed to decide which path each flow should use.

A widely used approach is ECMP (Equal-Cost Multi-Path) routing [2]. ECMP computes a hash over packet-header fields that identify a flow, commonly including the source and destination addresses, source and destination ports, and protocol number. The hash result is then mapped to one of the available equal-cost next hops.

Because packets belonging to the same flow produce the same hash value, they are normally forwarded along the same path. This flow-level path selection is simple and helps avoid packet reordering caused by changing paths within a flow, which is particularly useful for the transport model described earlier.

However, ECMP does not explicitly consider the current traffic load on each path. Its effectiveness therefore depends on how the set of active flows happens to be distributed by the hash function. When many independent flows are present, their aggregate traffic can often be spread reasonably well across the available paths [5]. With a smaller number of large or highly synchronized flows, however, an unlucky assignment may cause several flows to share the same links while other paths remain less utilized.

This distinction is particularly relevant to collective communication, where flows are generated in a structured and synchronized manner rather than independently. When examining the workload in this assignment, it may therefore be useful to ask not only *which paths are available*, but also *how the active flows are distributed across those paths*.

3 The Problem

We now bring the previous components together and examine the workload used in this assignment.

Consider a "multiple-1D-ring AllReduce" operating on 64 MB of data across 32 hosts. The hosts are organized into eight independent logical rings, with four hosts in each ring. The physical network is a two-tier spine–leaf topology consisting of four ToR switches and four spine switches. Each ToR connects eight hosts, and all network links operate at 10 Gbps. We will first examine the logical communication generated by the collective, then map this communication onto the physical network, and finally derive an idealized estimate of the AllReduce completion time.

3.1 Logical traffic transmission

The 32 hosts are divided into eight independent 1D rings, each containing four hosts. For each ring, the four participating hosts are placed in four different pods, with exactly one host from the ring in each pod.

During every communication step, each host sends one chunk of data to the next host in its logical ring while receiving from the previous host. Because consecutive hosts in every ring belong to different pods, all of these transfers are inter-pod.

With eight rings executing concurrently, each pod therefore generates eight outgoing inter-pod flows and receives eight incoming inter-pod flows during each communication step. Fig. 8 illustrates the logical ring placement and communication pattern.

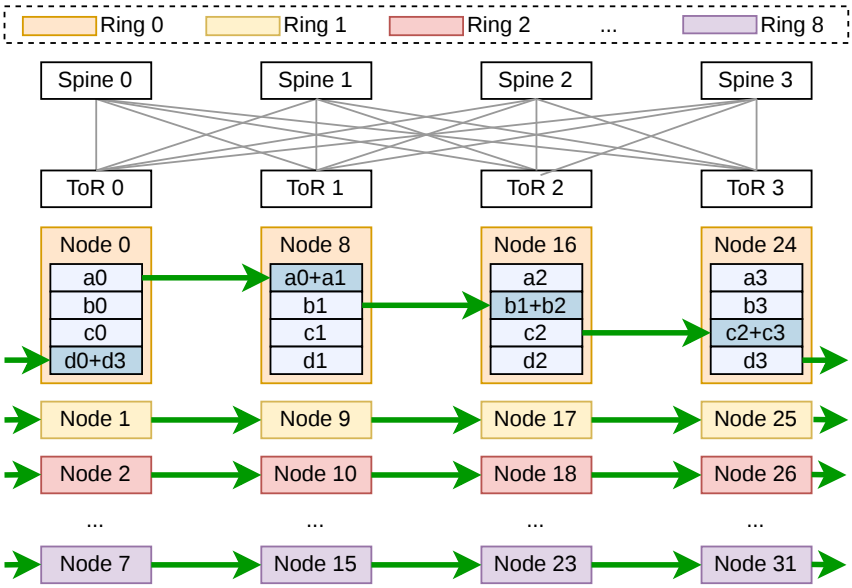


Figure 8: Logical placement and communication pattern of the multiple-1D-ring workload. Each ring contains one host from each of the four pods.

3.2 Mapping Logical Traffic to the Physical Network

The logical ring specifies which hosts communicate, but the resulting packets must ultimately traverse the physical spine–leaf network.

For an inter-pod flow, traffic travels from the source host through its ToR switch, across one of the spine switches, and then through the destination ToR switch to the destination host. As discussed earlier, ECMP determines which of the available equal-cost paths is selected for each flow. Fig. 9 illustrates this mapping.

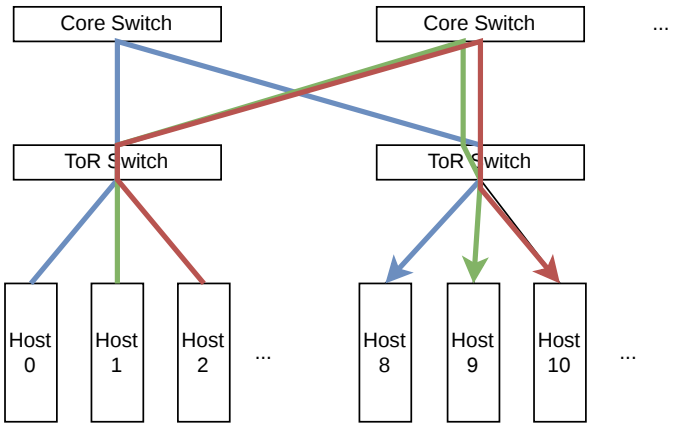


Figure 9: Example mapping of inter-pod flows onto paths in the physical spine–leaf network.

Between a pair of pods, the four spine switches provide four equal-cost paths. At each communication step, eight flows leave each pod. If these flows were distributed evenly across the four spine links, each link would carry two flows. Under this idealized assumption, the 10 Gbps link capacity would be shared equally by the two flows, giving each flow an effective throughput of approximately 5 Gbps.

3.3 Ideal Execution Time

For a four-host Ring AllReduce, the 64 MB collective is divided into four chunks of 16 MB each. Under the idealized traffic distribution described above, each flow receives approximately 5 Gbps of bandwidth. Ignoring propagation, processing, queueing, and protocol overheads, transmitting one 16 MB chunk would therefore take

$$\frac{16 \text{ MB} \times 8}{5 \text{ Gbps}} = 25.6 \text{ ms}.$$

As discussed in Fig. 4, a four-node Ring AllReduce consists of three ReduceScatter steps followed by three AllGather steps, for a total of six communication steps. The idealized completion time is therefore

$$6 \times 25.6 \text{ ms} = 153.6 \text{ ms}.$$

Thus, under *perfect* path balancing and ignoring other sources of overhead, we would expect the AllReduce to complete in approximately 153.6 ms.

3.4 What Happens in Simulation?

The calculation above is deliberately idealized. It assumes, among other things, that the active flows are evenly distributed across the available paths and that each pair of competing flows shares link capacity perfectly.

Now run the provided workload in the simulator and compare the measured completion time with this estimate. Does the simulated network behave as the simple model predicts?

If not, use the mechanisms introduced earlier in this document to investigate where the difference comes from. The baseline execution result will also be collected as part of Milestone 1 to verify that your simulation environment is configured correctly.

References

[1] Astra-sim. https://astra-sim.github.io/.

[2] Equal cost multipath load sharing - hardware ecmp. https://docs.nvidia.com/networking-ethernet-software/cumulus-linux-41/Layer-3/Equal-Cost-Multipath-Load-Sharing-Hardware-ECMP/.

[3] Ns-3 tutorial. https://www.nsnam.org/.

[4] Pipeline parallelism. https://www.deepspeed.ai/tutorials/pipeline/.

[5] Theophilus Benson, Ashok Anand, Aditya Akella, and Ming Zhang. Understanding data center traffic characteristics. *ACM SIGCOMM*, 2010.

[6] Sally Floyd, Dr. K. K. Ramakrishnan, and David L. Black. The Addition of Explicit Congestion Notification (ECN) to IP, 2001.

[7] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, Cedell Alexander, Eric Spada, James Dinan, Jeff Hammond, and Torsten Hoefler. Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms, 2025.

[8] Dong Lin and Robert Morris. Dynamics of random early detection. In *ACM SIGCOMM*, 1997.

[9] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in tensorflow, 2018.

[10] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for large-scale rdma deployments. In *ACM SIGCOMM*, 2015.